

Lab Manual — Week 5

Concurrency & Distributed Systems — Lebanese University, Faculty of Engineering

Topic: `java.util.concurrent`, Pools, and Backpressure

Learning Objectives

By the end of Week 5, you will understand:

1. **Backpressure:** Why unbounded queues are dangerous and how bounded queues enforce safety.
2. **Thread Pools:** How `ExecutorService` manages workers, queues, and task lifecycle.
3. **Task Submission:** The difference between `Runnable` and `Callable`, and how `Future` retrieves results.
4. **Rejection Policies:** What happens when a pool is saturated (worker threads full, queue full).
5. **Atomics:** Lock-free counters using compare-and-swap (CAS) to avoid contention.
6. **Read-Write Locks:** Optimizing read-heavy workloads where readers don't block each other.
7. **ThreadLocal:** Thread-scoped context storage, cleanup hazards, and use in thread pools.

Required Course Artifacts

This lab is not only a coding exercise. It also trains the written evidence expected in Checkpoint 2.

- **Concurrency scorecard:** complete the Week 5 `CONCURRENCY_SCORECARD.md`.
- **Checkpoint rubric:** use `CHECKPOINT2_RUBRIC.md` for the thread-pooled server deliverable.
- **Pitfalls Release 2:** review `PITFALLS.md`, especially oversized pools, unbounded queues, and nested task submission.
- **Explanation quality rubric:** for each answer, state the symptom, mechanism, fix, and remaining tradeoff.
- **Measurement template:** use `MEASUREMENT_TEMPLATE.md` for throughput and latency observations.
- **W4 to W5 continuity:** visibility and safe publication from Week 4 are still required when worker threads share state.

Lab Structure

Week 5 is organized into 5 parts, each with 2–3 demos and 1 exercise:

Part	Topic	Demos + Exercise
Part 1	BlockingQueue and Backpressure	Demo01–Demo03, Ex1
Part 2	Thread Pools and Task Submission	Demo04–Demo06, Demo14–Demo17, Ex2
Part 3	Atomics and CAS	Demo07–Demo09, Ex3
Part 4	Read-Write Lock	Demo10–Demo11, Demo18, Ex4
Part 5	ThreadLocal	Demo12–Demo13, Demo19, Ex5

1 Part 1: BlockingQueue and Backpressure

1.1 Demo01: Unbounded Queue Risk

Goal: Show why unbounded queues are dangerous.

Logic:

- Creates a `ConcurrentLinkedQueue` (unbounded by default).
- Enqueues 10,000 items.
- No backpressure: all items accepted instantly.
- Risk: if producers outpace consumers, memory fills until OOM.

Key Insight: Unbounded queues are unsafe. They give fast acks to producers while the system silently fails. Tail latency explodes long before OOM.

1.2 Demo02: Bounded BlockingQueue

Goal: Introduce the bounded queue solution.

Logic:

- Creates an `ArrayBlockingQueue` with fixed capacity (e.g., 2 items).
- First `offer()` succeeds.
- Second `offer()` returns `false` (queue is full).
- Demonstrates backpressure: queue rejects excess items.

Key Insight: Bounded queues enforce a contract: when full, the producer must handle rejection (block, reject, or drop). This is explicit and safe.

1.3 Demo03: Producer/Consumer Backpressure

Goal: Show the pattern: producers block, consumers drain.

Logic:

- Producer tries `put(item)` on a bounded queue.
- If full, producer blocks until consumer removes an item.
- Consumer calls `take()`, which blocks until an item is available.
- Result: natural flow control. Producers can't flood the system.

Code Pattern:

```
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(capacity);
queue.put(task); // Blocks if queue is full
Task t = queue.take(); // Blocks if queue is empty
```

1.4 Exercise 1: BoundedBuffer

Goal: Build a thread-safe bounded buffer using `BlockingQueue`.

Requirements:

- Accept a capacity parameter.
- Implement `put()` and `get()` methods.
- Block producers when full; block consumers when empty.
- Demonstrate correct blocking semantics in tests.

2 Part 2: Thread Pools and Task Submission

2.1 Demo04: FixedThreadPool

Goal: Introduce the standard thread pool.

Logic:

- Creates `Executors.newFixedThreadPool(2)`: 2 workers.
- Submits 5 tasks (e.g., increment a counter).
- Tasks queue up; workers dequeue and execute.
- Demonstrates task batching and lifecycle.

Key Insight: Fixed pools create N worker threads once; they reuse threads for all tasks. Much cheaper than creating a thread per task.

2.2 Demo05: BoundedExecutor

Goal: Build a custom bounded executor with rejection policy.

Logic:

- Manually construct `ThreadPoolExecutor` with:
 - Core and max worker threads.
 - `ArrayBlockingQueue` with fixed capacity.
 - `AbortPolicy`: throw `RejectedExecutionException` when queue is full.

Key Insight: You can customize executors. Rejection policies let you decide what happens when the pool is saturated.

2.3 Demo06: RejectionPolicyDemo

Goal: Show pool saturation and fail-fast rejection.

Logic:

- Build a pool with one worker and one queue slot.
- Submit one task that occupies the worker.
- Submit one task that fills the queue.
- Submit a third task and observe `RejectedExecutionException`.
- The demo uses `AbortPolicy`; other policies are lecture discussion.

Key Insight: Saturation must be explicit. A bounded pool either blocks upstream, rejects, or applies a chosen overload policy.

2.4 Demo14: Callable and Future

Goal: Task results via `Callable` and `Future`.

Logic:

- `Runnable`: no return value.
- `Callable<T>`: returns T, throws checked exceptions.
- `submit(Callable)` returns `Future<T>`.
- `future.get()` blocks until result available.

- Exceptions wrapped in `ExecutionException`.

Code Pattern:

```
Future<Integer> f = executor.submit(() -> computeSquare(7));
try {
    int result = f.get(); // Blocks until done
} catch (ExecutionException e) {
    // Handle computation exception
}
```

2.5 Demo15: CachedThreadPool

Goal: Contrast: unbounded threads, timeout-based cleanup.

Logic:

- `Executors.newCachedThreadPool()`: threads created on demand.
- Idle threads killed after 60 seconds.
- Good for bursty workloads; threads reuse for many tasks.
- Risk: unlimited thread creation if bursts are too large.

Key Insight: Fixed pools bound concurrency upfront. Cached pools are flexible but can explode.

2.6 Demo16: Fixed vs Cached Throughput

Goal: Measure I/O-like tasks: which pool is faster?

Logic:

- Submit tasks that simulate I/O (sleep + work).
- Fixed pool: workers reuse efficiently; throughput stable.
- Cached pool: fast initial response; throughput varies.
- Measure completion time and worker utilization.

Key Insight: Pool choice matters. Fixed pools are predictable; cached pools are reactive.

2.7 Measurement Table Template

Use the shared `MEASUREMENT_TEMPLATE.md`. For this lab, record at least:

Config	Workload	Warm-up	Runs	p50 / p99	Throughput
Fixed pool	I/O-like tasks				
Cached pool	I/O-like tasks				
Bounded pool	Saturation test				

Methodology:

- Run at least one warm-up before recording values.
- Repeat each measurement several times.
- Prefer p50 and p99 over only average latency.
- Record machine, JDK version, pool size, queue size, and task count.
- Treat `System.nanoTime()` as a simple lab tool, not a full microbenchmark replacement.

2.8 Demo17: Hand-Rolled Executor Contrast (Instructor Discussion)

Goal: Why standard thread pools are better than custom clones.

Logic:

- Implements a toy executor: `BlockingQueue`, `Semaphore`, workers.
- Shows race conditions, type unsafety, shutdown complexity.
- Demonstrates why `ThreadPoolExecutor` is the canonical solution.
- Included for guided discussion; it is not part of the student exercise contract.

Key Insight: Standard APIs are battle-tested. Hand-rolled concurrency hides subtle bugs.

2.9 Rejection Policy Decision Matrix

Policy	Behavior	Use only when
<code>AbortPolicy</code>	Throws <code>RejectedExecutionException</code> .	The caller must see overload and fail fast.
<code>CallerRunsPolicy</code>	Runs the task in the submitting thread.	Slowing the producer is acceptable backpressure.
<code>DiscardPolicy</code>	Silently drops the new task.	The task is optional telemetry or cache refresh work.
<code>DiscardOldestPolicy</code>	Drops the oldest queued task and retries.	Newer work is more valuable than stale queued work.

Key Insight: Rejection is not just a Java setting. It is a business decision about what the system should do under overload.

2.10 Exercise 2: `BoundedExecutor`

Goal: Build an executor that rejects excess tasks safely.

Requirements:

- Accept worker count and queue capacity.
- Use `ArrayBlockingQueue` and rejection policy.
- Verify that tasks are accepted up to capacity.
- Verify that excess tasks are rejected.

3 Part 3: Atomics and Compare-And-Swap

3.1 Demo07: Synchronized Counter

Goal: Baseline: lock-based counter.

Logic:

- Every method is **synchronized**.
- All threads compete for the lock.
- High contention under many concurrent increments.

Key Insight: Synchronization is simple but slow under contention. Can we do better?

3.2 Demo08: Atomic Counter

Goal: Lock-free counter using `AtomicInteger`.

Logic:

- `AtomicInteger.incrementAndGet()` uses CAS internally.
- No locks; threads spin-retry on conflict.
- Faster under low-to-medium contention.

Code Pattern:

```
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();
int val = counter.get();
```

3.3 Demo18: Writer Delay and Fairness Risk

Goal: Surface the liveness risk behind read-write locks.

Logic:

- Start several readers repeatedly acquiring the read lock.
- Submit a writer and measure how long it waits.
- Compare the meaning of unfair and fair `ReentrantReadWriteLock`.
- Fair locks can reduce barging, but may reduce throughput.

Key Insight: Read-write locks are a tradeoff: read throughput can improve while writer latency becomes the liveness question.

3.4 Demo09: CAS Retry Loop

Goal: Understand CAS semantics explicitly.

Logic:

- `AtomicInteger.compareAndSet(expect, update)`.
- If current value == `expect`, update to `update`.
- Retry loop until success.
- Shows why CAS is "optimistic": assume no conflict, retry if wrong.

Code Pattern:

```
AtomicInteger val = new AtomicInteger(0);
boolean success = val.compareAndSet(0, 1);
// Retry if false
while (!val.compareAndSet(oldVal, newVal)) {
    oldVal = val.get();
}
```

3.5 Exercise 3: AtomicMetrics

Goal: Build a metrics collector using atomics.

Requirements:

- Track count, sum, min, max of observed values.
- Use `AtomicInteger` or `AtomicLong` for thread safety.
- Allow concurrent updates without locks.
- Verify correctness under contention.

4 Part 4: Read-Write Lock

4.1 Demo10: Synchronized Read-Mostly Cache

Goal: Baseline: all access synchronized.

Logic:

- Every `get()` and `put()` synchronized.
- Even concurrent readers block each other.
- Inefficient for read-heavy workloads.

Key Insight: If most traffic is reads, full synchronization wastes concurrency.

4.2 Demo11: ReadWriteLock Cache

Goal: Optimize: concurrent readers, exclusive writers.

Logic:

- `ReentrantReadWriteLock`: read lock and write lock.
- Multiple threads can hold read lock simultaneously.
- Write lock is exclusive (no readers, no other writers).
- Benefit: read-heavy workloads scale much better.

Code Pattern:

```
ReentrantReadWriteLock lock = new ReentrantReadWriteLock();

// Read
lock.readLock().lock();
try {
    V val = map.get(key);
} finally {
    lock.readLock().unlock();
}

// Write
lock.writeLock().lock();
try {
    map.put(key, value);
} finally {
    lock.writeLock().unlock();
}
```

4.3 Exercise 4: ReadMostlyCache

Goal: Build a cache optimized for read-heavy access.

Requirements:

- Use `ReentrantReadWriteLock`.
- Support `get()`, `put()`, `invalidate()`.
- Verify reads don't block each other.
- Verify writes exclude all other access.

5 Part 5: ThreadLocal

5.1 Demo12: RequestContext with ThreadLocal

Goal: Store request-specific data per thread.

Logic:

- `ThreadLocal<T>`: each thread has its own instance.
- Perfect for request context (user ID, correlation ID, etc.).
- `set()`, `get()`, `remove()`.

Code Pattern:

```
private static ThreadLocal<String> requestId
    = ThreadLocal.withInitial(() -> "");

requestId.set("req-123");
String id = requestId.get();
requestId.remove(); // Important!
```

Key Insight: `ThreadLocal` is perfect for thread-per-request models (servlets, HTTP handlers).

5.2 Demo13: ThreadLocal Leak in Pools

Goal: The pitfall: threads are reused in pools.

Logic:

- Thread pool reuses threads for many tasks.
- If Task A sets `ThreadLocal.set(value)`, and never calls `remove()`,
- Then Task B runs in the same thread and sees the stale value.
- Risk: cross-contamination, information leaks.

Key Insight: `ThreadLocal` in pools is dangerous. Always call `remove()` in a `finally` block, or use `try-with-resources`.

5.3 Demo19: Measured ThreadLocal Memory Retention

Goal: Make the `ThreadLocal` leak concrete.

Logic:

- Use a single pooled worker.
- Store a byte array in a `ThreadLocal`.
- Run a later task on the same worker and observe retained bytes.
- Repeat with `remove()` and observe zero retained bytes.

Key Insight: In a pool, the thread outlives the request. If the `ThreadLocal` value is not removed, the value can outlive the request too.

5.4 Exercise 5: RequestContext with Cleanup

Goal: Build a safe request context wrapper.

Requirements:

- Wrap `ThreadLocal` to enforce cleanup.
- Provide `run(Runnable)` or `call(Callable)` that auto-cleans.

- Verify that values don't leak to subsequent tasks in a thread pool.

Summary: Bounded Concurrency Design

Week 5 teaches **bounded concurrency**: predictable systems under load.

1. **Backpressure**: Queues are bounded. Producers block or reject.
2. **Pools**: Workers are fixed. Tasks queue; workers reuse.
3. **Atomics**: Counters are lock-free. CAS handles contention.
4. **RW Locks**: Reads scale; writes exclude.
5. **ThreadLocal**: Context per thread; cleanup in pools.

All together: **safe, predictable, scalable systems under load**.

Checkpoint 2 Tie-In

Week 5 exercises directly support Checkpoint 2: a thread-pooled HTTP server.

- **Ex1 (BoundedBuffer)**: core of request queueing.
- **Ex2 (BoundedExecutor)**: the server's thread pool.
- **Ex3 (AtomicMetrics)**: server observability (request counts, latencies).
- **Ex4 (ReadMostlyCache)**: caching frequently-accessed data.
- **Ex5 (RequestContext)**: per-request isolation and cleanup.

Build these exercises, then apply them to the server checkpoint.

Checkpoint 2 Rubric Summary

Checkpoint 2 is worth 15% of the portfolio. The detailed rubric is in `CHECKPOINT2_RUBRIC.md`. The grading areas are:

Area	Weight	Evidence
Bounded request queue	20%	Queue capacity and full behavior Worker count, queue size, rejection/backpressure
Bounded executor policy	20%	
Correctness under contention	20%	Race-free metrics and shared state Blocking, rejection, starvation risks
Liveness and fairness notes	15%	
Measurement evidence	15%	Table, workload, p50/p99 or equivalent, tradeoff paragraph
Request context cleanup	10%	No ThreadLocal leak across pooled workers

Coming Up

Week 6 — Fork/Join and granularity: After bounding request execution, we ask a different question: when is CPU-bound work large enough to split? Fork/Join introduces work stealing and a break-even point where decomposition begins to pay.

Week 7 — Async design: After classical pools and Fork/Join, we move to `CompletableFuture`, time-out/fallback design, fan-out/fan-in, and testing strategies for asynchronous code.

Further Reading

- Brian Goetz et al., *Java Concurrency in Practice*, Chapters 6–9.
- Java API documentation for the executor, queue, read-write lock, and `ThreadLocal` APIs.